Backtesting Lab — Real Worked Example (BTC, 3 months, Momentum)

A single backtest computed **by hand, end to end**, from the raw database rows to every number the UI prints — then checked against the engine and against an independent NumPy/SciPy implementation.

Companion to `BACKTESTING_QUANT_FORMULAS.md`, which defines the formulas. This document applies them to one concrete run so the output can be verified rather than trusted.

---

0. The configuration

Control	Value
Asset universe	**BTC** (single asset → weight 100%)
Period	**3 months** → `PERIOD_DAYS["3M"] = 91` calendar days
Currency	USD
Initial capital	**$10,000**
Strategy	**Momentum**
Rebalance	**Monthly**
Benchmark	**GFI.JO** (Gold Fields, JSE, ZAR-denominated)
Fees	10 bps → `0.0010`
Slippage	5 bps → `0.0005`
Risk-free	**10.0%** → `0.10`

⚠ Reproducibility depends on the data window

Every number below was computed against the database as of **2026-08-18**, where the newest bar for both BTC and GFI.JO is **2026-08-17**.

The period is a **trailing** 91 days anchored on the newest available bar. The moment the crypto cron writes a 2026-08-18 bar, the window slides, the intersection changes, and **every figure in this document shifts**. If the site disagrees with the table in §12, check the last date on the chart *first* — a mismatched window is far more likely than a mismatched formula.

---

1. Why this configuration is unusually good for verification

Momentum on a single asset has a closed-form solution. The strategy computes:

$$w_i = \frac{\max(0, \mu_i)}{\sum_j \max(0, \mu_j)}, \qquad \text{fallback to equal-weight if } \sum_j \max(0, \mu_j) = 0$$

With $n = 1$ there are only two branches, and **both return the same answer**:

- $\mu > 0 \rightarrow w = [\mu/\mu] = [1]$
- $\mu \leq 0 \rightarrow$ denominator is $0 \rightarrow$ fallback `equalWeight(1)` = $[1]$

So $w = [1]$ on every single rebalance, regardless of what BTC did. Three consequences that make the whole run hand-checkable:

1. **Weights never change after the opening trade** → every subsequent rebalance has $\sum_i |w_i^{\text{new}} - w_i^{\text{old}}| = 0$ → **zero cost**, and total fees are exactly the opening trade.

2. **Cash is always 0** → the 10% risk-free rate never touches the equity curve. It affects only Sharpe, Sortino and Alpha.
3. **The equity curve is a scalar multiple of the BTC price series.**

Verifying branch 2 is actually taken. The drift is negative at the first two rebalances and positive at the third — so both code paths are exercised, and both produce $w = [1]$:

Rebalance	Date	Estimation window (backward-looking)	μ (annualized)	Branch
1	2026-06-17	2026-05-18 … 2026-06-15 (21 obs)	**−2.040204**	fallback → $[1]$
2	2026-07-01	2026-06-01 … 2026-06-30 (21 obs)	**−1.975903**	fallback → $[1]$
3	2026-08-03	2026-07-03 … 2026-07-31 (21 obs)	**+0.216337**	positive → $[1]$

Note each window **ends the bar before** the rebalance date — the look-ahead guarantee.

---

2. Step 1 — Raw data from `historical_prices`

The engine reads `adj_close ?? close` for every symbol in one batched, paginated query.

Symbol	Rows	First bar	Last bar	`adj_close` populated
BTC	4,353	2014-09-17	2026-08-17	4,353 / 4,353
GFI.JO	4,237	2010-01-01	2026-08-17	4,237 / 4,237

---

3. Step 2 — FX conversion (and why it cannot affect the result)

GFI.JO is a JSE listing stored in **ZAR**. `usdConverterForMarkets` divides by the current rate:

$$P^{\text{USD}} = \frac{P^{\text{ZAR}}}{16.2057}$$

Last bar: $\text{ZAR}\ 673.98 \div 16.2057 = \$41.589071$.

> **The same current rate is applied to every historical bar** — there is no historical FX series. That is a real simplification, but for **this** backtest it is provably harmless, because the benchmark enters only through its *returns*, and a constant scale factor cancels:
>
> $$r_t = \frac{P_t/k - P_{t-1}/k}{P_{t-1}/k} = \frac{P_t - P_{t-1}}{P_{t-1}}$$
>
> Verified numerically: $\max_t |r_t^{\text{USD}} - r_t^{\text{ZAR}}| = 1.9 \times 10^{-16}$ (floating-point noise). **Beta, alpha, tracking error and information ratio are all FX-invariant here.** The rate would matter if GFI.JO were a *held position* rather than the benchmark, since then its USD value would be mixed with BTC's in the weighted math.

BTC needs no conversion (crypto is USD-native).

---

4. Step 3 — Alignment and the 91-day trim

BTC quotes every calendar day; GFI.JO only on JSE trading days. The engine intersects them:

$$\mathcal{D} = \text{dates}(\text{BTC}) \cap \text{dates}(\text{GFI.JO})$$

then keeps the most recent 91 calendar days.

Aligned bars	**64**
Window	**2026-05-18 → 2026-08-17**
Calendar span	**91 days**
Coverage	`truncated: false` — the full 3 months was available

91 calendar days collapse to 64 bars — BTC's weekend prints are discarded because GFI.JO has no price on those days. This is correct behaviour, not data loss: you cannot mark a JSE stock to market on a Sunday.

BTC in the aligned window: first `$76,954.171875` (2026-05-18), last `$64,316.98828125`.

---

5. Step 4 — Periods per year

Inferred from actual date spacing, **not** assumed:

$$\overline{\Delta d} = \frac{91}{64 - 1} = 1.4444444444 \text{ days}$$

$$P = \frac{365.25}{1.4444444444} = \mathbf{252.8653846154}$$

A JSE-calendar series, exactly as expected — **not** 365, because the intersection dropped the weekend crypto bars. Had BTC been backtested against a crypto benchmark, P would be ≈ 365 and every annualized figure below would be ~20% larger.

---

6. Step 5 — Estimation window and warm-up

$$L = \min(252,\ \max(20,\ \lfloor 64/3 \rfloor)) = \min(252, \max(20, 21)) = \mathbf{21}$$

$$\text{start} = \min(21,\ \max(1,\ 64 - 2)) = \mathbf{21}$$

The first 21 bars are consumed estimating the first allocation and never appear on the curve:

Aligned window	2026-05-18 → 2026-08-17 (64 bars)
Backtest window	**2026-06-17 → 2026-08-17 (43 bars, 42 returns)**

> **Expect the chart to start 2026-06-17, not 2026-05-18.** A "3 month" backtest displays about **2 months** of curve. This is disclosed in the header, not hidden.

Shrinkage (irrelevant here — Momentum never builds a covariance matrix, but the header still prints it):

$$\delta = \text{clip}\left(\frac{1 \times 2/2}{21},\ 0.05,\ 0.9\right) = \text{clip}(0.047619, \ldots) = \mathbf{0.05} \rightarrow \text{"}5\backslash\%\text{"}$$

Header should read: *43 days simulated · 3 trades* and *estimation window: 21 bars, 3 rebalances, 5% shrinkage*.

---

7. Step 6 — The opening trade

From flat to fully invested:

$$c = |1 - 0| \times (0.0010 + 0.0005) = \mathbf{0.0015}$$

$$V_0 = 10{,}000 \times (1 - 0.0015) = \mathbf{\$9{,}985.00}$$

$$\text{fees paid} = \$15.00$$

Every later rebalance is free, because w never changes (§1):

$$c = |1 - 1| \times 0.0015 = 0 \qquad \text{at 2026-07-01 and 2026-08-03}$$

$$\boxed{\text{Fees Paid} = \$15.00 \text{ exactly}}$$

Turnover (one-way): $\frac{1}{2}(1.0) + 0 + 0 = \mathbf{0.500}$.

---

8. Step 7 — The equity curve, in closed form

With $w = 1$, cash $= 0$ and no further costs, the recursion $V_{t+1} = V_t(1 + r_{\text{BTC},t})$ telescopes:

$$\boxed{V_t = 9{,}985 \times \frac{P_t^{\text{BTC}}}{P_0^{\text{BTC}}}}$$

Verified against the step-by-step engine recursion: max absolute difference 7.3×10^{-12} across all 43 bars — floating-point noise only.

Final value:

$$V_{42} = 9{,}985 \times \frac{64{,}316.98828125}{64{,}418.4453125} = 9{,}985 \times 0.9984250314 = \mathbf{\$9{,}969.2739}$$

Chart footer should read: *Final: $9,969*.

Sample of the curve:

Date	BTC close	Equity
2026-06-17	64,418.4453	$9,985.00
2026-06-18	62,896.4727	$9,749.09
2026-06-30	58,558.8594	$9,076.75 ← trough
2026-07-01	60,003.7578	$9,300.71
2026-07-21	66,505.1250	$10,308.44 ← peak
2026-07-31	62,813.7461	$9,736.27
2026-08-03	63,460.8984	$9,836.58
2026-08-17	64,316.9883	$9,969.27

---

9. Step 8 — Rebalance calendar and trade legs

Monthly buckets change at index 10 (July) and index 33 (August):

$$\text{rebalance indices} = [0,\ 10,\ 33] \to [\text{2026-06-17},\ \text{2026-07-01},\ \text{2026-08-03}]$$

Leg boundaries append the final bar: $[0, 10, 33, 42]$ → **3 trade legs**.

Quantity is constant across all three legs

$$q_e = \frac{V_{t_0} \cdot w}{P_{t_0}} = \frac{9{,}985 \cdot (P_{t_0}/P_0)}{P_{t_0}} = \frac{9{,}985}{P_0} = \frac{9{,}985}{64{,}418.4453} = \mathbf{0.15500219}$$

The price cancels — a direct consequence of a single, always-fully-invested asset. All three legs report the same quantity, which is a useful sanity check on screen.

Leg 1 — 2026-06-17 → 2026-07-01

$$\text{fees} = (64{,}418.4453 + 60{,}003.7578) \times 0.15500219 \times 0.0015 = 124{,}422.2031 \times 0.15500219 \times 0.0015 = \mathbf{\$28.9286}$$

$$\text{PnL} = (60{,}003.7578 - 64{,}418.4453) \times 0.15500219 - 28.9286 = -684.2862 - 28.9286 = \mathbf{-\$713.2148}$$

$$\text{PnL\%} = \frac{-713.2148}{64{,}418.4453 \times 0.15500219} = \frac{-713.2148}{9{,}985} = \mathbf{-7.1429\%}$$

Status **loss**. Duration: 14 **calendar** days.

Leg 2 — 2026-07-01 → 2026-08-03

$$\text{fees} = (60{,}003.7578 + 63{,}460.8984) \times 0.15500219 \times 0.0015 = \mathbf{\$28.7059}$$

$$\text{PnL} = (63{,}460.8984 - 60{,}003.7578) \times 0.15500219 - 28.7059 = 535.8643 - 28.7059 = \mathbf{+\$507.1584}$$

$$\text{PnL}\% = \mathbf{+5.4529\%}$$

Status **win**. Duration: 33 days.

Leg 3 — 2026-08-03 → 2026-08-17

$$\text{fees} = (63{,}460.8984 + 64{,}316.9883) \times 0.15500219 \times 0.0015 = \mathbf{\$29.7088}$$

$$\text{PnL} = (64{,}316.9883 - 63{,}460.8984) \times 0.15500219 - 29.7088 = 132.6958 - 29.7088 = \mathbf{+\$102.9870}$$

$$\text{PnL}\% = \mathbf{+1.0470\%}$$

Status **win**. Duration: 14 days.

> **Per-leg fees total** $87.34, but Fees Paid reports **$15.00.** Both are correct — they are different accounting bases. §I.4.0 of the formula reference charges each leg a notional round trip at entry and exit prices (an attribution view), while §I.2.24 charges actual portfolio weight changes. Since the weights never changed, only the opening trade was a real transaction. **$15.00 is the authoritative figure**; the per-leg number double-counts holds that were never actually traded.

---

10. Step 9 — Every metric, computed by hand

Working values across the 42 returns:

$$\bar{r} = 0.0001118102, \qquad s(r) = 0.0175006296, \qquad \sqrt{P} = 15.9017415592$$

$$\bar{r} \cdot P = 0.0282729261 \quad (+2.83\% \text{ annualized arithmetic mean})$$

10.1 Total Return

$$\frac{9{,}969.2739}{9{,}985.00} - 1 = \mathbf{-0.0015749686} \to \mathbf{-0.16\%}$$

> **This equals BTC's raw price return over the window**, $64{,}316.99 / 64{,}418.45 - 1$, because the $15 opening fees cale s V_0$ and every later value by the same factor and cancels in the ratio. The fee was genuinely paid—it is why the curve starts at 9,985 — but it does not appear in Total Return. **Do not use Total Return to check whether fees were charged;** use the $10,000 \to$9,985 gap.

10.2 CAGR

$$y = \frac{43 - 1}{252.8653846} = 0.1660962811 \text{ years}$$

$$\text{CAGR} = 0.9984250314^{\,1/0.1660962811} - 1 = 0.9984250314^{\,6.02059} - 1 = \mathbf{-0.0094448526} \to \mathbf{-0.94\%}$$

10.3 Annualized Volatility

$$\sigma = 0.0175006296 \times 15.9017415592 = \mathbf{0.2782904885} \to \mathbf{27.83\%}$$

10.4 Sharpe — where the 10% risk-free rate bites

$$\text{numerator} = 0.0282729261 - 0.10 = \mathbf{-0.0717270739}$$

$$S = \frac{-0.0717270739}{0.2782904885} = \mathbf{-0.2577417369} \to \mathbf{-0.26}$$

The strategy made money on an arithmetic basis (+2.83% annualized) and still scores a negative Sharpe, purely because the hurdle is 10%. At $R_f = 0$ the Sharpe would be $0.0282729 / 0.2782905 = +0.102$. The risk-free input is doing all the sign-flipping here — a good thing to be aware of when reading this run.

10.5 Sortino

20 of the 42 returns are negative.

$$\sum_t \min(r_t, 0)^2 = 0.0061797123$$

$$\sigma_d = \sqrt{\frac{0.0061797123}{42}} \times 15.9017415592 = 0.0121299 \times 15.9017416 = \mathbf{0.1928875399}$$

$$\text{Sortino} = \frac{-0.0717270739}{0.1928875399} = \mathbf{-0.3718595505} \to \mathbf{-0.37}$$

Note the denominator divides by **42** (all periods), not by 20 (the losing periods).

10.6 Max Drawdown

Peak stays at $9,985 (the opening value) for the entire first month:

$$\text{MDD} = \frac{9{,}076.7514 - 9{,}985.00}{9{,}985.00} = \mathbf{-0.0909613063} \to \mathbf{-9.10}\%$$

Reached on **2026-06-30**.

10.7 Average Drawdown

$$\overline{\text{DD}} = \frac{-1.3795708907}{43} = \mathbf{-0.0320830440} \to \mathbf{-3.21}\%$$

10.8 Calmar

$$\frac{-0.0094448526}{0.0909613063} = \mathbf{-0.1038337400} \to \mathbf{-0.10}$$

10.9 VaR 95%

$n = 42$, so $h = 41 \times 0.05 = 2.05$, $\lfloor h \rfloor = 2$, $\lceil h \rceil = 3$.

Sorted returns (first four): $-0.0295334629,\ -0.0294418200,\ -0.0266944245,\ -0.0262647507$

$$Q = -0.0266944245 + 0.05 \times (-0.0262647507 - (-0.0266944245))$$

$$= -0.0266944245 + 0.05 \times 0.0004296738 = \mathbf{-0.0266729409} \to \mathbf{-2.67}\%$$

Confirmed identical to `numpy.quantile(r, 0.05)`.

10.10 CVaR 95%

Returns at or below −0.0266729409 — exactly three of them:

$$\frac{-0.0266944245 + (-0.0294418200) + (-0.0295334629)}{3} = \mathbf{-0.0285565681} \to \mathbf{-2.86}\%$$

10.11 Skewness

$$g_1 = \frac{1}{42} \sum_t \left(\frac{r_t - \bar{r}}{s} \right)^3 = \mathbf{+0.0991235845} \to \mathbf{0.099}$$

Mildly right-skewed.

10.12 Kurtosis (excess)

$$g_2 = \frac{1}{42}\sum_t \left(\frac{r_t - \bar{r}}{s}\right)^4 - 3 = \mathbf{-0.7076080856} \to \mathbf{-0.708}$$

Thinner-tailed than Gaussian over this particular two-month window — unusual for BTC, and a direct consequence of the sample being short and containing no crash.

10.13 Beta vs GFI.JO

$$\sum_t (r_t - \bar{r})(r_{m,t} - \bar{r_m}) = 0.0065442149, \qquad \sum_t (r_{m,t} - \bar{r_m})^2 = 0.0569457195$$

$$\beta = \frac{0.0065442149}{0.0569457195} = \mathbf{0.1149202256} \to \mathbf{0.11}$$

Near-zero, as expected: Bitcoin and a South African gold miner have almost nothing in common.

10.14 Annualized Alpha

$$\bar{R_m} \cdot P = 0.2602896662 \quad (\text{GFI.JO } +26.03\% \text{ annualized})$$

$$\alpha = 0.0282729261 - [0.10 + 0.1149202256 \times (0.2602896662 - 0.10)]$$

$$= 0.0282729261 - [0.10 + 0.0184205246] = \mathbf{-0.0901475985} \to \mathbf{-9.01\%}$$

The 10% risk-free rate dominates: with $\beta \approx 0.11$, the benchmark contributes almost nothing, so alpha is essentially "return minus the hurdle".

10.15 Tracking Error

$$\overline{(r - r_m)} = -0.0009175504, \qquad s(r - r_m) = 0.0370939621$$

$$\text{TE} = 0.0370939621 \times 15.9017415592 = \mathbf{0.5898585992} \to \mathbf{58.99\%}$$

Enormous — which is the honest answer when the "benchmark" is uncorrelated with the portfolio. **A 59% tracking error is a signal that GFI.JO is not a meaningful benchmark for BTC.**

10.16 Information Ratio

$$\text{IR} = \frac{-0.0009175504 \times 252.8653846}{0.5898585992} = \frac{-0.2320125}{0.5898586} = \mathbf{-0.3933429815} \to \mathbf{-0.393}$$

10.17 Trade statistics

$$\text{Win Rate} = \frac{2}{3} = \mathbf{0.6666666667} \to \mathbf{66.67\%}$$

$$\text{Profit Factor} = \frac{507.1584 + 102.9870}{713.2148} = \frac{610.1454}{713.2148} = \mathbf{0.8554862}$$

$$\text{Avg Win} = \frac{610.1454}{2} = \mathbf{\$305.0727}, \qquad \text{Avg Loss} = -\frac{713.2148}{1} = \mathbf{-\$713.2148}$$

$$\text{Avg Duration} = \frac{14 + 33 + 14}{3} = 20.33 \to \mathbf{20}$$

> **Win rate 67% with profit factor 0.86.** Two wins and one loss, but the single loss was larger than both wins combined. This is exactly why the two metrics are reported separately — win rate alone would suggest a profitable strategy.

10.18 Best / Worst Day

$$\max_t r_t = +\mathbf{0.0436540899} \to +\mathbf{4.37}\% \quad (\text{2026-07-13} \to \text{2026-07-14})$$

$$\min_t r_t = -\mathbf{0.0295334629} \to -\mathbf{2.95}\% \quad (\text{2026-07-30} \to \text{2026-07-31})$$

---

11. Independent verification — three implementations agree

Computed three ways: the production TypeScript engine, the closed-form hand derivation above, and an independent NumPy/SciPy script written from the formulas alone.

Metric	TypeScript engine	NumPy / SciPy	|diff|
totalReturn	−0.00157497	−0.00157497	1.4e−09
cagr	−0.00944485	−0.00944485	2.6e−09
annualVol	0.27829049	0.27829049	1.5e−09
sharpe	−0.25774174	−0.25774174	3.1e−09
sortino	−0.37185955	−0.37185955	5.0e−10
calmar	−0.10383374	−0.10383374	0.0e+00
maxDrawdown	−0.09096131	−0.09096131	3.7e−09
avgDrawdown	−0.03208304	−0.03208304	4.0e−09
var95	−0.02667294	−0.02667294	9.0e−10
cvar95	−0.02855657	−0.02855657	1.9e−09
skew	0.09912358	0.09912358	4.5e−09
kurt	−0.70760809	−0.70760809	4.4e−09
beta	0.11492023	0.11492023	4.4e−09
alpha	−0.09014760	−0.09014760	1.5e−09
trackingError	0.58985860	0.58985860	8.0e−10
informationRatio	−0.39334298	−0.39334298	1.5e−09
winRate	0.66666667	0.66666667	0.0e+00
profitFactor	0.85548624	0.85548624	0.0e+00
avgWin	305.07272298	305.07272300	2.0e−08
avgLoss	−713.21479449	−713.21479400	4.9e−07
bestDay	0.04365409	0.04365409	1.0e−10
worstDay	−0.02953346	−0.02953346	2.9e−09
turnover	0.50000000	0.50000000	0.0e+00
feesPaid	15.00000000	15.00000000	0.0e+00

24 / 24 agree. The largest relative difference is 8.9×10^{-7}, entirely explained by rounding the TypeScript values to 8 decimals when exporting them for comparison — not by any computational disagreement.

The NumPy check used `numpy.quantile` (Hyndman–Fan type 7) for VaR, which the engine matches by construction, and `numpy.std(ddof=1)` for the sample standard deviation.

---

12. What you should see on the site

Run the configuration in §0 and compare. **Check the chart's last date is 2026-08-17 first.**

Header

```
43 days simulated · 3 trades
estimation window: 21 bars, 3 rebalances, 5% shrinkage
```

Results tab — 12 cards

Card	Expected	Badge
TOTAL RETURN	**−0.16%**	negative
CAGR	**−0.94%**	negative
ALPHA	**−9.01%**	negative
BETA	**0.11**	negative*
SHARPE	**−0.26**	negative
SORTINO	**−0.37**	negative
CALMAR	**−0.10**	negative
MAX DRAWDOWN	**−9.10%**	negative
ANNUAL VOL.	**27.83%**	negative*
VAR 95%	**−2.67%**	negative
CVAR 95%	**−2.86%**	negative
WIN RATE	**66.67%**	positive

* Beta's badge is driven by `beta − 1` and Annual Vol's by `−vol`, so both read "negative" by design regardless of the value. See §I.1.4 and §I.1.9 of the formula reference.

Equity chart footer: `Final: $9,969`

Advanced metrics tab — 24 rows

Row	Expected
Total Return	−0.16%
Annualized Return (CAGR)	−0.94%
Annualized Volatility	27.83%
Sharpe Ratio	−0.258
Sortino Ratio	−0.372
Calmar Ratio	−0.104
Max Drawdown	−9.10%
Average Drawdown	−3.21%
Skewness	0.099
Kurtosis (excess)	−0.708
VaR 95%	−2.67%
CVaR 95%	−2.86%
Beta vs Benchmark	0.115
Annualized Alpha	−9.01%
Tracking Error	58.99%
Information Ratio	−0.393
Win Rate	66.67%
Profit Factor	0.855
Average Win	$305.07
Average Loss	$-713.21
Best Day	4.37%
Worst Day	−2.95%
Turnover (one-way)	0.500
Fees Paid	$15.00

Exposure is intentionally absent (see §I.2.25). Average Loss renders as `$-713.21` — the `usd()` helper puts the sign after the dollar sign.

Trades tab — 7 tiles

Tile	Expected
Number of trades	3
Winners	2
Losers	1
Avg win	$305.07
Avg loss	$713.21
Profit factor	0.86
Avg duration	20

Trades table

Entry	Exit	Side	Qty	Entry price	Exit price	PnL	PnL %	Duration	Fees	Status
2026-06-17	2026-07-01	BUY	0.155002…	$64,418.45$|60,003.76	-713.21| $-$ 7.1428.93	loss				
2026-07-01	2026-08-03	BUY	0.155002…	$60,003.76$|63,460.90	507.16|5.4528.71	win				
2026-08-03	2026-08-17	BUY	0.155002…	$63,460.90$|64,316.99	102.99|1.0529.71	win				

All three quantities identical — see §9.

Curves tab

- **Drawdown:** minimum −9.10% at 2026-06-30. Returns to 0% (a new high-water mark) on 2026-07-14, 2026-07-20 and 2026-07-21.
- **Rolling volatility (30):** exactly **13** plotted points (42 returns − 30 + 1), first one at **2026-07-29**. No down-sampling applies (`step = floor(43/200) = 1`).
- **Rolling Sharpe (60): completely empty.** 42 returns cannot fill a 60-period window. The chart correctly renders nothing rather than a fabricated ramp — this is the null-until-full policy working, not a broken chart.
- **Allocation over time:** a single flat 100% BTC band.
- **Monthly heatmap:** see §13 — **this one does not reconcile.**

---

13. Finding: the monthly heatmap does not reconcile with Total Return

Running this backtest surfaced a genuine defect.

What the heatmap shows:

Month	Bars used	Equity	Return
2026-06	2026-06-17 … 2026-06-30	$9,985.00$ →9,076.75	**−9.0961%**
2026-07	2026-07-01 … 2026-07-31	$9,300.71$ →9,736.27	**+4.6830%**
2026-08	2026-08-03 … 2026-08-17	$9,836.58$ →9,969.27	**+1.3490%**
		YTD	**−3.5554%**

But Total Return is −0.1575%. The YTD column is wrong by **3.40 percentage points** over three months.

Cause

`src/utils/backtesting.ts:537` computes each month as

$$R_{y,m} = \frac{V_{\text{last bar of month}}}{V_{\text{first bar of month}}} - 1$$

The return **from the previous month's last bar to this month's first bar is dropped** — it belongs to neither bucket. Here that discards two real returns:

Dropped link	Return
2026-06-30 → 2026-07-01	**+2.467429%**
2026-07-31 → 2026-08-03	**+1.030272%**
Compounded	**+3.523122%**

Chaining them back in closes the gap exactly:

$$(1 - 0.0355536)(1 + 0.0352312) - 1 = \mathbf{-0.00157497} \;=\; \text{Total Return} \quad ✓$$

Fix

A month's return should be measured against the **previous month's close**, not its own first bar:

$$R_{y,m} = \frac{V_{\text{last bar of month } m}}{V_{\text{last bar of month } m-1}} - 1$$

With that definition:

Month	Corrected
2026-06	−9.0961%
2026-07	**+7.2660%** (was +4.6830%)
2026-08	**+2.3932%** (was +1.3490%)
YTD	**−0.157497%** ✓ reconciles exactly

Severity

Every month after the first is understated (or overstated) by exactly the gap return. The error **compounds across the row**, so it grows with the number of months displayed — a 5-year backtest would show a YTD column that is materially wrong every year, in a way that looks plausible.

The bug does not touch any other metric: Total Return, CAGR, drawdown, Sharpe and everything else are computed from the full return series and are unaffected. **It is isolated to the heatmap.**

Not fixed — flagged only, since this document's job was to verify. One-line change in the monthly-bucket loop.

---

14. Summary

- **24 / 24 metrics reproduce exactly** across the production engine, a closed-form hand derivation, and an independent NumPy/SciPy implementation. Largest relative disagreement 8.9×10^{-7}, attributable entirely to export rounding.
- **Fees, warm-up, look-ahead and the rebalance calendar all behave as specified.** Fees are exactly $15.00; the estimation window is 21 bars and strictly backward-looking; three rebalances fire on the correct calendar boundaries.
- **One real defect found**: the monthly-returns heatmap drops the inter-month return, so its YTD column does not reconcile with Total Return (§13).
- **Two things that look wrong but are correct**: the empty Rolling Sharpe chart (42 returns cannot fill a 60-period window) and the chart starting a month after the requested period (warm-up).
- **One configuration caveat worth surfacing to users**: a 58.99% tracking error against GFI.JO says the benchmark is uncorrelated with the portfolio. The number is right; the benchmark choice is what should be reconsidered.